HDOC Day-66

Recursive Multiplication of Two Square Matrices in C

Problem Statement

Write a C program to multiply two square matrices using recursion. Do not use iterative loops for the multiplication logic. Read the order and both matrices, calculate A x B recursively, and display the product matrix.

Algorithm

1. Start and read matrix order n. Validate that 1 <= n <= 8.

2. Declare two n x n integer matrices A and B and a long long result matrix C.

3. Read A and B row-wise using loops; initialize each element of C to zero.

4. Call multiplyRecursive(n, A, B, C, 0, 0, 0).

5. If row i equals n, return (all entries completed).

6. If column j equals n, recurse to the first column of the next row.

7. If k equals n, recurse to the next column in the current row.

8. Otherwise add A[i][k] * B[k][j] to C[i][j] and recurse with k + 1.

9. Display C row-wise and stop.

Test Case Table

For matrices, a slash (/) separates rows. Enter the values row-wise with spaces or new lines. Every test case is an independent program run.

TC	Order n	Matrix A	Matrix B	Expected result
1	2	1 2 / 3 4	5 6 / 7 8	19 22 / 43 50
2	1	7	6	42
3	2	1 0 / 0 1	9 8 / 7 6	9 8 / 7 6
4	2	0 0 / 0 0	1 2 / 3 4	0 0 / 0 0
5	2	-1 2 / 3 -4	5 -6 / -7 8	-19 22 / 43 -50
6	2	2 0 / 0 3	4 5 / 6 7	8 10 / 18 21
7	3	1 0 0 / 0 1 0 / 0 0 1	1 2 3 / 4 5 6 / 7 8 9	1 2 3 / 4 5 6 / 7 8 9
8	2	1 2 / 3 4	0 1 / 1 0	2 1 / 4 3
9	0	Not entered	Not entered	Invalid matrix order. Enter 1 to 8.
10	9	Not entered	Not entered	Invalid matrix order. Enter 1 to 8.

C Program

```
#include <stdio.h>

#define MAX 8
```

```
void multiplyRecursive(int n, int A[n][n], int B[n][n],
                       long long C[n][n], int i, int j, int k)
{
    if (i == n)
        return;
    if (j == n) {
        multiplyRecursive(n, A, B, C, i + 1, 0, 0);
        return;
    }
    if (k == n) {
        multiplyRecursive(n, A, B, C, i, j + 1, 0);
        return;
    }
    C[i][j] += (long long)A[i][k] * B[k][j];
    multiplyRecursive(n, A, B, C, i, j, k + 1);
}

int main(void)
{
    int n, i, j;
    printf("Enter order of square matrices (1 to %d): ", MAX);
    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        printf("Invalid matrix order. Enter 1 to %d.\n", MAX);
        return 1;
    }
    int A[n][n], B[n][n];
    long long C[n][n];
    printf("Enter %d elements of matrix A (row-wise):\n", n*n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (scanf("%d", &A[i][j]) != 1) {
                printf("Invalid matrix element.\n");
                return 1;
            }
    printf("Enter %d elements of matrix B (row-wise):\n", n*n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (scanf("%d", &B[i][j]) != 1) {
                printf("Invalid matrix element.\n");
                return 1;
            }
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            C[i][j] = 0;
    multiplyRecursive(n, A, B, C, 0, 0, 0);
    printf("Product matrix A x B:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("%lld%s", C[i][j], j == n-1 ? "" : " ");
        printf("\n");
    }
    return 0;
}
```

Compilation and Execution

Save as hdoc66.c. In Kali Linux:

```
gcc  hdoc66.c -o hdoc66

./hdoc66
```

Compilation and Execution - Screenshot

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc66.c

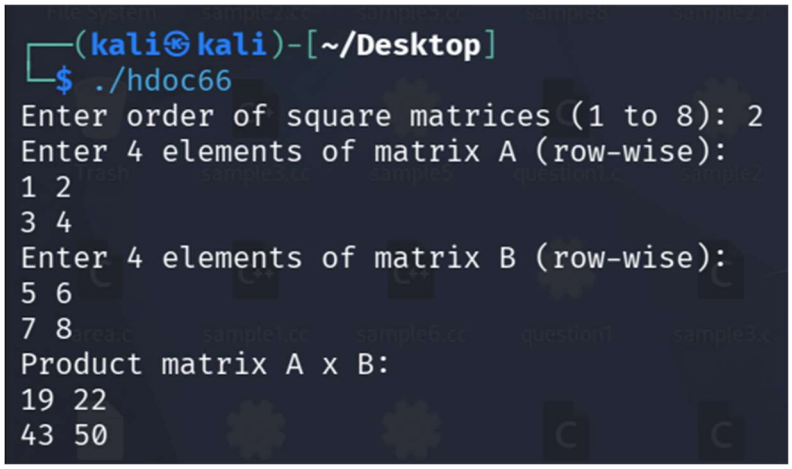
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc66.c -o hdoc66
```

Test Case 1 - Screenshot

Scenario: Standard 2x2 multiplication. Order: 2.

Matrix A: 1 2 / 3 4 Matrix B: 5 6 / 7 8

Expected output: 19 22 / 43 50

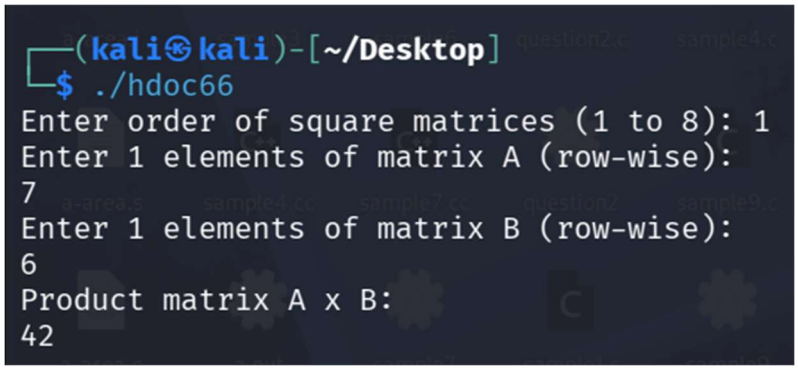


Test Case 2 - Screenshot

Scenario: Minimum valid order. Order: 1.

Matrix A: 7 Matrix B: 6

Expected output: 42

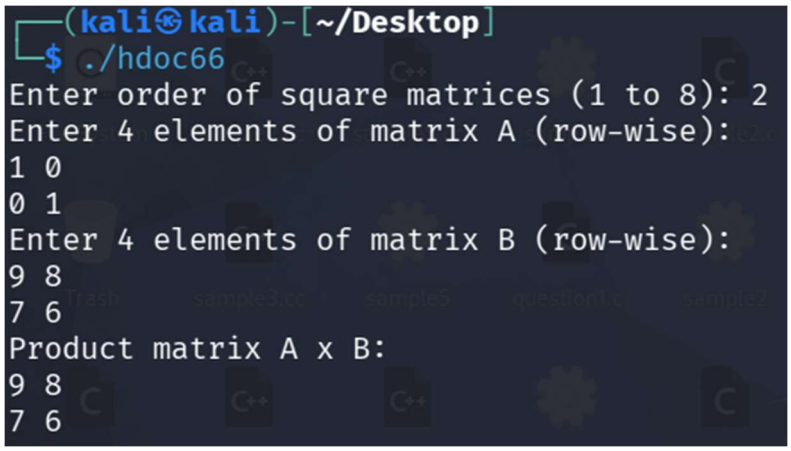


Test Case 3 - Screenshot

Scenario: Identity matrix. Order: 2.

Matrix A: 1 0 / 0 1 Matrix B: 9 8 / 7 6

Expected output: 9 8 / 7 6

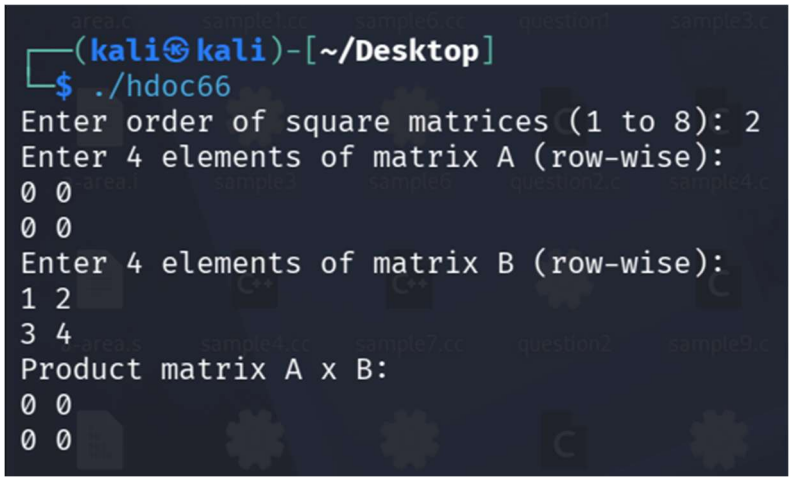


Test Case 4 - Screenshot

Scenario: Zero matrix. Order: 2.

Matrix A: 0 0 / 0 0 Matrix B: 1 2 / 3 4

Expected output: 0 0 / 0 0



Test Case 5 - Screenshot

Scenario: Negative elements. Order: 2.

Matrix A: -1 2 / 3 -4 Matrix B: 5 -6 / -7 8

Expected output: -19 22 / 43 -50

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc66
Enter order of square matrices (1 to 8): 2
Enter 4 elements of matrix A (row-wise):
-1 2
3 -4
Enter 4 elements of matrix B (row-wise):
5 -6
-7 8
Product matrix A x B:
-19 22
43 -50
```

Test Case 6 - Screenshot

Scenario: Diagonal matrix. Order: 2.

Matrix A: 2 0 / 0 3 Matrix B: 4 5 / 6 7

Expected output: 8 10 / 18 21

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc66
Enter order of square matrices (1 to 8): 2
Enter 4 elements of matrix A (row-wise):
2 0
0 3
Enter 4 elements of matrix B (row-wise):
4 5
6 7
Product matrix A x B:
8 10
18 21
```

Test Case 7 - Screenshot

Scenario: 3x3 identity. Order: 3.

Matrix A: 1 0 0 / 0 1 0 / 0 0 1 Matrix B: 1 2 3 / 4 5 6 / 7 8 9

Expected output: 1 2 3 / 4 5 6 / 7 8 9

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc66
Enter order of square matrices (1 to 8): 3
Enter 9 elements of matrix A (row-wise):
1 0 0
0 1 0
0 0 1
Enter 9 elements of matrix B (row-wise):
1 2 3
4 5 6
7 8 9
Product matrix A x B:
1 2 3
4 5 6
7 8 9
```

Test Case 8 - Screenshot

Scenario: Column swap matrix. Order: 2.

Matrix A: 1 2 / 3 4 Matrix B: 0 1 / 1 0

Expected output: 2 1 / 4 3

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc66
Enter order of square matrices (1 to 8): 2
Enter 4 elements of matrix A (row-wise):
1 2
3 4
Enter 4 elements of matrix B (row-wise):
0 1
1 0
Product matrix A x B:
2 1
4 3
```

Test Case 9 - Screenshot

Scenario: Invalid order. Order: 0.

Expected output: Invalid matrix order. Enter 1 to 8.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc66
Enter order of square matrices (1 to 8): 0
Invalid matrix order. Enter 1 to 8.
```

Test Case 10 - Screenshot

Scenario: Out-of-range order. Order: 9.

Expected output: Invalid matrix order. Enter 1 to 8.

